NanoBook: Phase 2 Report

The fast book · 18 October 2026 · C++23, GCC 14 / Clang 18, Linux · Phase 2 complete

1. Summary

Phase 2 builds a second order book, `FastBook`, with **exactly the same behaviour** as the phase 1 reference book but built for speed: no heap allocation while trading, and data laid out so the CPU cache can do its job. The reference book stays unchanged as the answer key.

Status	**Complete.** All 8 steps done, and both exit criteria met (§5.4).
Allocations per added order	**2.01 → 0** (reference book → fast book)
Zero-allocation test, whole fast book	**Passes**: 100,000 random operations after a warm-up, exactly 0 allocations (exit criterion 2)
Speed (same session)	**2.2–5.3× faster** than the reference book on every benchmark; the tick-array experiment up to 16× on a large book
Correctness	All **96 book and exchange tests** run on both fast-book variants; in the exit check the differential fuzzer agreed with the reference book on **162 million** random operations (exit criterion 1)
Tests	**373** in Release builds (362 with sanitizers), on GCC and Clang
Planted bugs	**12** deliberate bugs across the new code, every one caught

2. The plan and where it stands

Step	What	Status
0	Design note (`docs/fast-book-design.md`)	✓ Done
1	Zero-allocation test tool (a counting `operator new`)	✓ Done
2	Differential fuzzer (`tools/nanobook_fuzz`)	✓ Done
3	Order pool and intrusive queue, with unit tests	✓ Done
4	Fast book v1: correct first, with temporary parts	✓ Done
5	Flat hash table (`IdTable`) replaces the temporary id lookup	✓ Done
6	Price levels: sorted array (main) and tick array (experiment)	✓ Done
7	Order layout: 48 vs 64 bytes per order, measured (48 kept)	✓ Done
8	Exit check: 10^7+ fuzz on several seeds, zero-allocation test, this report	✓ Done

The method throughout: build each part on its own with its own tests, put it into the book while everything else stays the same, check it with the fuzzer, then measure before against after in one session. A failure then always points at the one part that just changed.

3. What the fast book is made of



Part	File	Data structure	Key operations and cost
Order slot	`fast/order.hpp`	48-byte struct; links are `uint32_t` slot numbers, not pointers	—
Order pool	`fast/order_pool.hpp`	One array of `max_orders` slots + a **free list** threaded through the empty slots	take / give back a slot: **O(1)**, no allocation
Price-level queue	`fast/intrusive_queue.hpp`	**Intrusive doubly linked list**: the links live inside the orders; the queue keeps first, last, count (12 bytes)	add at back, remove from anywhere: **O(1)**
Level	`fast/level.hpp`	price + **cached total quantity** + queue (32 bytes)	level quantity: **O(1)** (no walk)
Id lookup	`fast/id_table.hpp`	**Open-addressing hash table**, Fibonacci hashing, linear probing, **backward-shift deletion**, ≥ 2× `max_orders` entries	find / insert / erase: **O(1)** expected, usually 1–2 reads
Price levels (main)	`fast/sorted_levels.hpp`	**Sorted array per side, best price at the end**	best: **O(1)**; find a price: **binary search O(log n)**; add/remove near best: few moves
Price levels (experiment)	`fast/tick_levels.hpp`	**Tick array** (one slot per price) + **two-level bitmap**	find a price: **O(1)**; next price: 64 at a time with `countr_zero`
The book	`fast/fast_book.hpp`	`BasicFastBook<Levels>`, a template over the level structure	matching, cancel, modify: same rules as the reference book

4. The data structures, one by one

4.1 Order pool and free list (step 3)

Like a car park with numbered spaces: every space is built once, at start-up, and orders take and return them.

```
start:        slot 0 → 1 → 2 → 3 → end          (free list; first orders sit side by side)
take 2:       [Alice] [Bob]  free: 2 → 3 → end
give back 0:  [free]  [Bob]  free: 0 → 2 → 3    (released last = reused first: still in cache)
```

- The free list costs no memory: it runs through the empty slots' own `next` field.
- A full pool is the contract's `BookFull`: no new behaviour.
- A free slot holds a special tag in `prev`, so Debug builds catch a slot released twice.

4.2 Intrusive queue (step 3)

Each price level's first-come-first-served line. The orders carry their own links; the level only remembers who is first and last.

```
level $10.00: first=0, last=1, count=2
   slot 0 Alice  (prev: none, next: 1)  ◄──►  slot 1 Bob  (prev: 0, next: none)
cancel Alice → the level's first becomes 1, Bob's prev becomes none. Two updates.
```

4.3 Flat hash table, `IdTable` (step 5)

Answers "which slot holds order #N?". A row of lockers, at least twice as many as orders.

Idea	What it does
Fibonacci hashing	home locker = (id × 11400714819323198485) keeping the top bits: spreads near-sequential exchange ids apart
Linear probing	home taken → try the next locker, wrapping at the end
Backward-shift deletion	after removing, later entries slide back into the hole **unless their home is after the hole**, so a search can never stop early at a gap
id 0 = empty	the exchange never issues id 0

Worked example (8 lockers, real hash values): ids #1, #9 and #17 all hash to locker 4.

```
after inserts:   locker 4: #1   5: #9   6: #3 (home 6)   7: #17
erase #9:        hole at 5 → #3 stays (home 6 is after the hole) → #17 slides to 5
result:          locker 4: #1   5: #17  6: #3            7: empty
without the slide, a search for #17 would stop at the empty locker 5 and wrongly answer "not here"
```

4.4 Price levels: sorted array, binary search (step 6, the main design)

```
bids (best at the end):   [ 990 | 998 | 1000 | 1001 ]      best bid = last element, O(1)
find 1000:  binary search: check the middle, keep the half it's in (about 7 looks for 100 levels)
new best 1002:  goes at the end, nothing moves
new deepest 980: goes at the front, every level shifts by one (rare, but slow in a deep book)
```

- Memory follows the number of levels in use, not the price band: any band works.
- Both arrays are reserved for `max_orders` levels at start-up, so they never grow while trading.

4.5 Price levels: tick array with a bitmap (step 6, the experiment)

```
band $9.90–$10.10:   slot = price − 990       (finding a level never searches)
bitmap:   bit per price, 1 = level exists      best bid = highest set bit (one CPU instruction)
summary:  bit per 64-price block                skips whole empty stretches
```

- Memory follows the **band width**: 64 KB per side for a $100 stock at ±10%, about 450 MB for BRK.A. So it accepts narrow bands only (≤ 65,536 ticks) and stays an experiment.

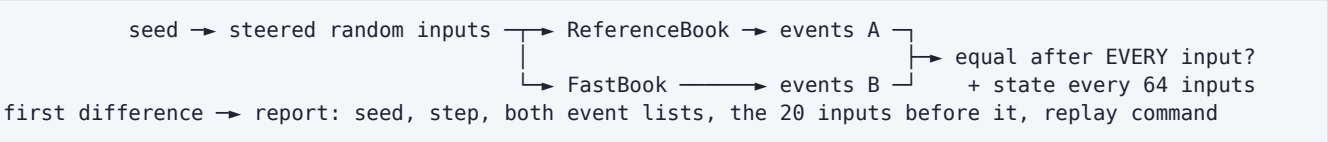
---

5. How correctness was proven

5.1 Four layers of checking

Layer	What it checks	Scale
Typed tests	Every book and exchange test runs on `ReferenceBook`, `FastBook` and `TickFastBook`	96 tests × 3 books
Unit tests per part	Pool, queue, hash table, both level structures, each on its own, including mini differential tests against `std::unordered_map` and `std::map`	50 tests
Differential fuzzer	Random steered inputs to the reference book and a fast book; every event compared after every input, the book state every 64 inputs, invariants every 1,024	162 million operations in the exit check (§5.4), both variants, including under ASan/UBSan
Zero-allocation tests	The counting `operator new`, checked in both directions, then the whole fast book: 100,000 operations after warm-up	Exactly 0 allocations

5.2 The differential fuzzer



Steered so the interesting cases are common: prices around a drifting middle, cancels and modifies that mostly hit resting orders, four owners (self-matches), every order type, edge values, and a tiny 16-order book for a quarter of the seeds. It uses `std::mt19937_64` output directly, so a seed replays identically with any compiler.

5.3 Planted bugs: do the tests have teeth?

Part	Planted bug	Caught by
Fuzzer	A book that drops an IOC's closing cancel	Fuzzer, step 11: "events differ"
Fuzzer	A book that reports a level quantity one too high	Fuzzer state check, step 64
Allocation counter	Never counts	5 tests
`EventBuffer`	`clear()` reallocates	Zero-allocation test
Pool	The same slot handed out twice	2 tests, plus the double-free check
Queue	Unlinking forgets a back link	5 tests
Fast book	A partial fill doesn't lower the level's cached total	18 tests; fuzzer step 64
Fast book	A modify that loses priority doesn't update `filled`	**1 test only; fuzzer step 561,789**
`IdTable`	Erase without backward shift	5 tests; fuzzer step 17,866
`IdTable`	Shift ignores homes	2 tests; fuzzer invariant check, step 2,048
Sorted levels	The search returns one position too early	14 tests; crash in Release, assert names the line in Debug
Tick levels	Summary bitmap never cleared	3 tests; crash in Release

The `filled` bug is the lesson of the phase: one test caught it, and the fuzzer needed 561,789 inputs. Either safety net alone could have missed it.

5.4 The exit check (step 8)

Phase 2's two exit criteria (README roadmap), run on the final code (commit `8e60edf`). The full output, with every command, is in `docs/data/phase2_exit_check.txt`.

Exit criterion	Run	Operations	Result
1. Differential fuzzer passes 10^7+ operations	`FastBook`, seeds 1–5, normal book sizes	5 × 10,000,000	✓ PASS
	`FastBook`, seeds 101–103, tiny 16-order book	3 × 10,000,000	✓ PASS
	`TickFastBook`, seeds 1–5, normal book sizes	5 × 10,000,000	✓ PASS
	`TickFastBook`, seeds 101–103, tiny 16-order book	3 × 10,000,000	✓ PASS
	Both books under ASan + UBSan, seed 7	2 × 1,000,000	✓ PASS, no memory errors
2. Zero allocations while trading	`nanobook_alloc_tests`: the counter checked both ways, then the whole `FastBook` and `TickFastBook` (100,000 random operations after a warm-up)		✓ 11 of 11 pass, exactly 0 allocations

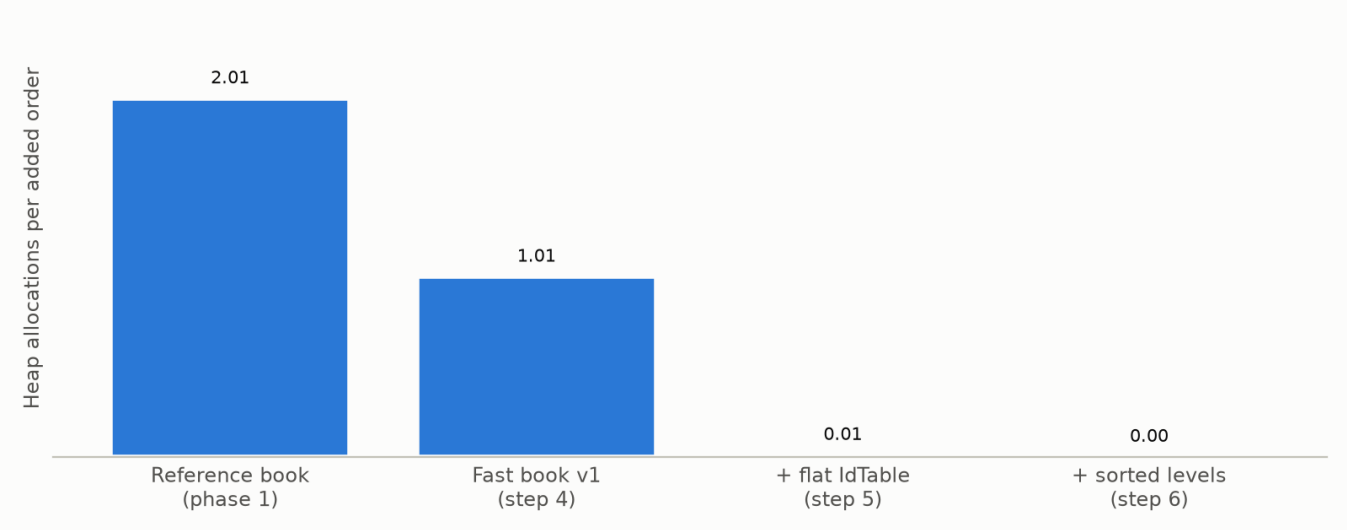
162 million random operations, zero differences from the reference book. The final code also passes all 373 tests on GCC and Clang (362 under the sanitizers), the format check and clang-tidy.

---

6. Performance

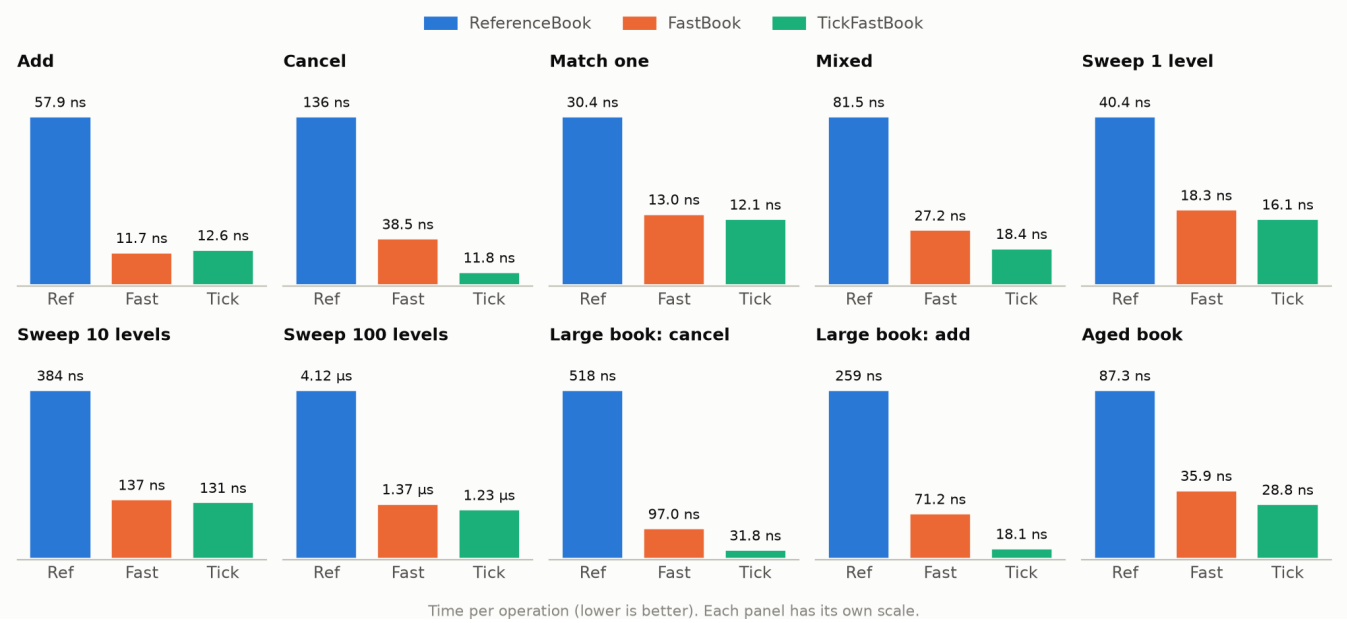
All numbers: Intel Core i7-14650HX, WSL2, GCC 14 `-O3`, pinned to one core. **For comparison only, never for publishing** (rule 20). Google Benchmark reports averages; tail latency comes in phase 3. Because this machine's speed drifts between sessions, every comparison below was measured **within one session** (the older build was rebuilt from git and run back to back). Full details: `docs/perf-log.md`.

6.1 Allocations: the main goal of phase 2

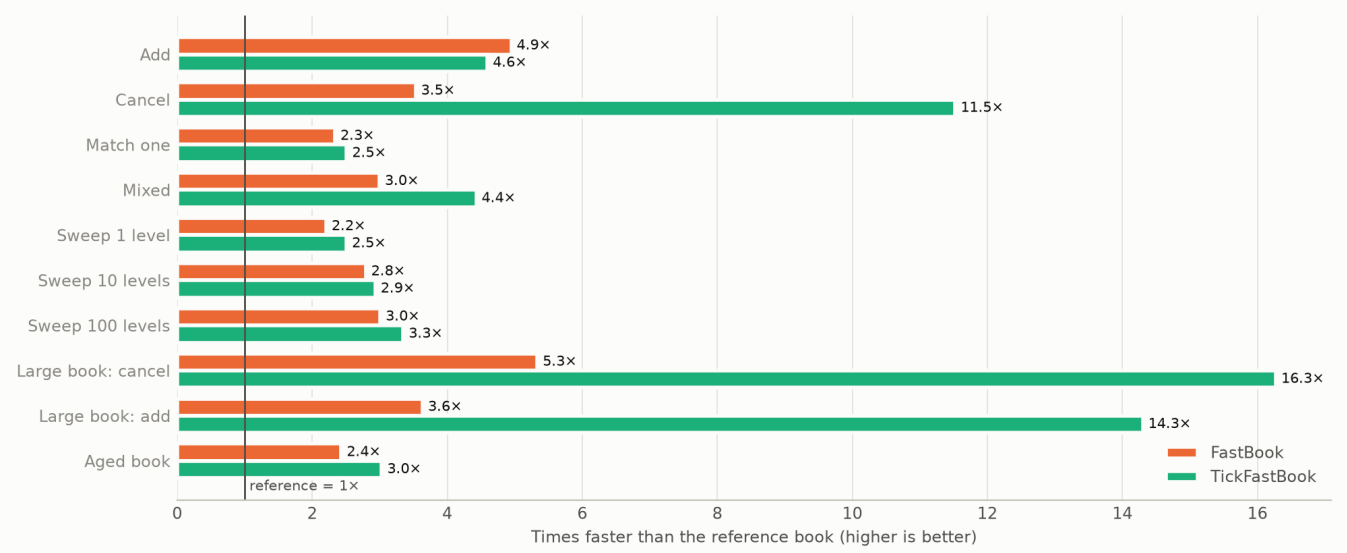


Heap allocations per added order at each step. The reference book allocates about twice per order (a list node and a map node); the fast book reaches zero.

6.2 Time per operation, all three books (one session)



Time per operation for all three books, one panel per benchmark, each with its own scale. The fast book is below the reference book in every panel; the tick-array experiment is lowest wherever a price has to be found (cancel, the large book).



How many times faster each fast book is than the reference book. FastBook is 2.2 to 5.3 times faster everywhere; the tick array reaches 11.5 times on cancel and 14 to 16 times on the large book.

Benchmark	Reference	FastBook	TickFastBook	Fast vs reference
Add a resting order	57.9 ns	11.7 ns	12.6 ns	**4.9×**
Cancel	136.0 ns	38.5 ns	11.8 ns	**3.5×**
Match one	30.4 ns	13.0 ns	12.1 ns	2.3×
Mixed stream	81.5 ns	27.2 ns	18.4 ns	**3.0×**
Sweep 1 / 10 / 100 levels	40 / 384 / 4,116 ns	18 / 137 / 1,368 ns	16 / 131 / 1,229 ns	2.2× / 2.8× / 3.0×
Large book: cancel	517.5 ns	97.0 ns	31.8 ns	**5.3×**
Large book: add	258.9 ns	71.2 ns	18.1 ns	3.6×
Aged book	87.3 ns	35.9 ns	28.8 ns	2.4×

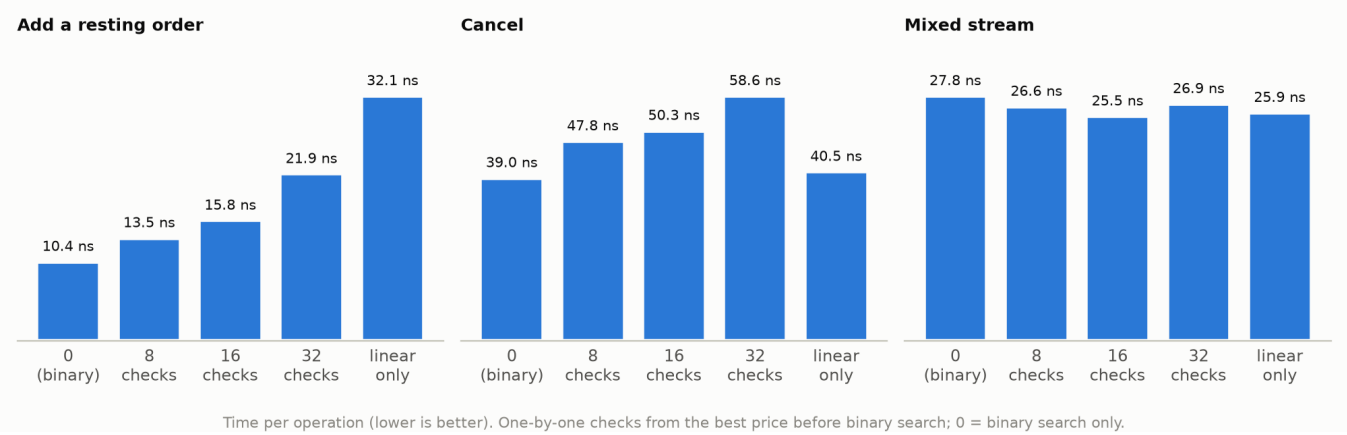
Allocations per operation: the reference book makes 2.01 per added order (1.26 per mixed operation, 0.90 in the aged book); **both fast books make 0 everywhere**.

What the table says:

- **The biggest gains are on the large book** (100,000 orders): too big for the CPU's fastest caches (L1 and L2), so the reference book's scattered tree nodes cost real memory trips, while the fast book's data sits in a few compact arrays.
- **The tick array wins wherever a price has to be found** (cancel, the large book: 3–4× faster than the sorted array), because a level is one array index away instead of a binary search. Where no lookup is needed (adding at a known level, matching, sweeping) the two are within ~10%.
- That meets the design's condition for the **moving-window tick array**, so it's planned for phase 6. `FastBook` keeps the sorted array for now, because it works for any price band.

6.3 The search inside the sorted levels: a hybrid measured and dropped

The design planned a hybrid: check the levels nearest the best price one by one, then switch to binary search. Measured in one session against plain binary search and a plain linear scan:



Hybrid search experiment. On add and cancel, which spread orders evenly over 100 levels, binary search alone is fastest and every one-by-one check slows it down. On the clustered mixed stream all variants are within about 9 percent.

The one-by-one phase never paid for itself, so `FastBook` uses plain binary search. Real Nasdaq order flow may cluster more tightly near the best price, so this is on the design note's revisit list for phase 4 (`docs/fast-book-design.md` §13).

6.4 Gains at each step (each measured in its own session)

Step	Before	After	Add	Cancel	Mixed	Allocations per add
4	Reference book	Fast v1 (pool, queues, cached totals)	3.0×	2.4×	~2.0×	2.01 → 1.01
5	Temporary `std::unordered_map` lookup	Flat `IdTable`	2.1×	1.4×	1.3×	1.01 → 0.01
6	Temporary `std::map` levels	Sorted array	≈ same	≈ same	≈ same	0.01 → **0**

Step 6's main speed gain was on sweeps (1.7–1.9×, no tree node freed per emptied level); on add, cancel and the mixed stream the difference was inside the noise, because a 100-level `std::map` stays in cache. Ratios from different sessions can't be multiplied into one overall number on this machine; §6.2 is the like-for-like comparison.

6.5 Order slot: 48 bytes packed vs 64 bytes padded (step 7)

At 48 bytes, 4 orders fill 3 cache lines, so half of them straddle two lines; padding to 64 (`alignas(64)`) makes each order exactly one line but uses a third more memory. Measured in one session on an idle machine, both fast books, all 10 benchmarks: every difference was within ±6%, mostly inside the noise, and the few beyond it pointed both ways. **No consistent winner, so the order stays at 48 bytes** (simpler, a quarter less memory). Even the large book's orders (about 5 MB) fit in this CPU's 30 MB L3 cache, so a book too big for L3, and phase 3's tail latency, are where it could still matter.

---

7. What phase 2 taught

Lesson	Where it came from
Build one part at a time	Every failure pointed at the part just swapped in
Two independent implementations	The fast book shares no logic with the reference book, so a shared bug can't hide from the fuzzer
Tests and fuzzing need each other	The `filled` bug: 1 test caught it; the fuzzer took 561,789 inputs
The compiler may delete unused allocations	Clang counted 1 where 2 were expected; `keep()` fixed the test
Return types: don't make the compiler guess too early	`auto*` helpers crashed Clang 18 and were refused by GCC
Done means every CI check	Skipping clang-tidy once turned CI red
Benchmark setup can dominate	Building a 10,000-level book worst-first took minutes; slow untimed setup also makes runs balloon
Measure in one session, one process per benchmark	The machine slowed 5× mid-run once; buffered output was lost when a run was killed
A sorted array has a worst case	A deep insert shifts every better level: a tail-latency risk for phase 3
Measure, don't assume	The hybrid search was a reasonable idea and lost to plain binary search
Benchmark only on an idle machine	A run was thrown away when the laptop was in use; correctness checks (fuzzing) don't need it idle
A tie is a result	48 vs 64 bytes: no consistent winner, so the simpler, smaller layout stays

---

8. What's next

1. **Phase 3, the benchmark harness:** an `rdtsc` timer for every single operation, histograms, and p50 / p99 / p99.9 latency, including the sorted array's deep-insert worst case. Published numbers need bare-metal Linux (rule 20), not WSL2; dual-booting Ubuntu on this laptop is the cheapest way, pinned to a performance core.
2. **Phase 4, ITCH:** replay a real Nasdaq trading day, and re-check the revisit list (design §13) with real order flow.
3. **Phase 6:** the moving-window tick array, now justified by the step 6 numbers (design §6.3).